



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie
Houari Boumediene
Faculté d'Informatique

Rapport de Projet

Cryptographie

Implémentation des algorithmes cryptographiques en Python

Réalisé par

Likou Yani Anis	232331741011
Daoud Omar Rayane	232332447215
Benchetioui Ahmed Ayoub	232331451018

Année universitaire : 2024/2025

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs du projet	4
1.3	Structure du projet	4
1.4	Comment exécuter	5
2	Chiffrements Classiques	6
2.1	Chiffrement de César	6
2.1.1	Principe	6
2.1.2	Code source	6
2.1.3	Fonctionnement	6
2.2	Chiffrement Affine	7
2.2.1	Principe	7
2.2.2	Code source	7
2.3	Chiffrement de Vigenère	7
2.3.1	Principe	7
2.3.2	Code source	8
2.4	Chiffrement de Hill	8
2.4.1	Principe	8
2.4.2	Code source	8
2.5	Chiffrement de Playfair	9
2.5.1	Principe	9
2.5.2	Code source	9
2.6	One-Time Pad (OTP)	9
2.6.1	Principe	9
2.6.2	Code source	9
2.7	Substitution Aléatoire	10
2.7.1	Principe	10
2.7.2	Code source	10
3	Outils de Cryptanalyse	11
3.1	Analyse Fréquentielle	11
3.1.1	Principe	11
3.1.2	Code source	11
3.2	Indice de Coïncidence	11
3.2.1	Principe	11
3.2.2	Code source	11
3.3	Test de Kasiski	12

3.3.1	Principe	12
3.3.2	Code source	12
4	Chiffrement Symétrique (Synchrone)	13
4.1	DES (Data Encryption Standard)	13
4.1.1	Principe	13
4.1.2	Architecture	13
4.1.3	Code source	13
4.1.4	Variantes implémentées	14
4.2	AES (Advanced Encryption Standard)	14
4.2.1	Principe	14
4.2.2	Code source	14
4.2.3	Modes implémentés	15
4.3	RC4 (Rivest Cipher 4)	15
4.3.1	Principe	15
4.3.2	Code source	15
4.3.3	Variantes implémentées	16
5	Chiffrement Asymétrique	17
5.1	RSA	17
5.1.1	Principe	17
5.1.2	Code source	17
5.2	ElGamal	18
5.2.1	Principe	18
5.2.2	Code source	18
5.3	Diffie-Hellman	18
5.3.1	Principe	18
5.3.2	Code source	18
5.4	Elliptic Curve Cryptography (ECC)	19
5.4.1	Principe	19
5.4.2	Code source	19
6	Fonctions de Hachage	20
6.1	Principe général	20
6.2	MD5	20
6.3	SHA-256 et SHA-512	20
6.4	HMAC	21
7	Signatures Numériques	22
7.1	Principe	22
7.2	Signature RSA	22
7.3	Signature ElGamal	22
7.4	Signature DSA	23
7.5	Signature ECDSA	23
7.6	Benchmark	23

8	Canal de Communication Sécurisé	24
8.1	Architecture	24
8.2	Génération des clés	24
8.3	Utilitaires cryptographiques	24
8.4	Protocole de communication	25
8.5	Exécution	25
9	Interface Graphique (GUI)	26
9.1	Présentation de l'application	26
9.2	Architecture de l'interface	26
9.3	Apport Pédagogique	27
10	Questions et Réponses	28
10.1	Limites pratiques de l'OTP (One-Time Pad)	28
10.2	Impact de la longueur de la clé et lien avec l'OTP	28
10.3	Vulnérabilité du chiffrement de Hill	28
10.4	Chiffrement RSA et Padding OAEP	29
10.5	Sélection de Rijndael pour l'AES face à Serpent	29
11	Conclusion	30
11.1	Résumé des implémentations	30
11.2	Compétences acquises	30

Chapitre 1

Introduction

1.1 Contexte

La cryptographie est la science de la protection de l'information par le biais de techniques mathématiques. Elle joue un rôle fondamental dans la sécurisation des communications numériques modernes, que ce soit pour le chiffrement des données, l'authentification des utilisateurs ou la vérification de l'intégrité des messages.

1.2 Objectifs du projet

Ce projet a pour objectif l'implémentation en Python des principaux algorithmes cryptographiques étudiés en cours, couvrant :

- Les **chiffrements classiques** : César, Affine, Vigenère, Hill, Playfair, OTP, Substitution aléatoire
- Les **outils de cryptanalyse** : Analyse fréquentielle, Indice de coïncidence, Test de Kasiski
- Le **chiffrement symétrique (synchrone)** : DES, 2DES, 3DES, DESX, AES (ECB/CBC/CTR), RC4
- Le **chiffrement asymétrique** : RSA, ElGamal, Diffie-Hellman, ECC
- Les **fonctions de hachage** : MD5, SHA-256, SHA-512, HMAC
- Les **signatures numériques** : RSA, ElGamal, DSA, ECDSA
- Un **canal de communication sécurisé** : Client/Serveur avec RSA + AES-GCM

1.3 Structure du projet

Le projet est organisé en répertoires thématiques :

```
Cryptography/  
  Classical Ciphers/      # Chiffrements classiques  
  Synchrone/             # Chiffrement symétrique (DES, AES, RC4)  
  asynchrone/            # Chiffrement asymétrique (RSA, ElGamal, DH)  
  Hashing/               # Fonctions de hachage  
  Signatures/            # Signatures numériques  
  secure_Channel/        # Canal sécurisé Client/Serveur
```

1.4 Comment exécuter

Chaque script est exécutable indépendamment :

```
1 python "Classical Ciphers/Cesar.py"  
2 python Synchrone/AES.PY  
3 python asynchrone/RSA.py
```

Exécution d'un script

Pour le canal sécurisé, lancer d'abord le serveur puis le client dans deux terminaux séparés.

Chapitre 2

Chiffrements Classiques

2.1 Chiffrement de César

2.1.1 Principe

Le chiffrement de César est un chiffrement par substitution mono-alphabétique. Chaque lettre du texte clair est décalée d'un nombre fixe de positions dans l'alphabet.

Formules :

$$\text{Chiffrement : } E(x) = (x + k) \mod 26 \quad (2.1)$$

$$\text{Déchiffrement : } D(x) = (x - k) \mod 26 \quad (2.2)$$

2.1.2 Code source

```
1 def cesar_encrypt(text: str, shift: int) -> str:
2     result = ""
3     for char in text:
4         if char.isalpha():
5             ascii_offset = ord('A') if char.isupper() else ord('a')
6             result += chr((ord(char) - ascii_offset + shift) % 26
7                           + ascii_offset)
8         else:
9             result += char
10    return result
11
12 def cesar_decrypt(text: str, shift: int) -> str:
13    return cesar_encrypt(text, -shift)
```

Listing 2.1 – Cesar.py - Fonctions principales

2.1.3 Fonctionnement

Le programme propose un menu interactif permettant de chiffrer ou déchiffrer un texte avec une clé entière positive. Le déchiffrement réutilise la fonction de chiffrement avec un décalage négatif.

2.2 Chiffrement Affine

2.2.1 Principe

Le chiffrement affine utilise une fonction affine pour le chiffrement :

$$E(x) = (a \cdot x + b) \mod 26 \quad (2.3)$$

$$D(y) = a^{-1} \cdot (y - b) \mod 26 \quad (2.4)$$

où a doit être premier avec 26 (i.e. $\gcd(a, 26) = 1$).

2.2.2 Code source

```

1 def affine_encrypt(text: str, a: int, b: int) -> str:
2     result = ""
3     for char in text:
4         if char.isalpha():
5             ascii_offset = ord('A') if char.isupper() else ord('a')
6             x = ord(char) - ascii_offset
7             y = (a * x + b) % 26
8             result += chr(y + ascii_offset)
9         else:
10            result += char
11    return result
12
13 def affine_decrypt(text: str, a: int, b: int) -> str:
14     try:
15         a_inv = pow(a, -1, 26)
16     except ValueError:
17         raise ValueError("L'inverse n'existe pas")
18     result = ""
19     for char in text:
20         if char.isalpha():
21             ascii_offset = ord('A') if char.isupper() else ord('a')
22             y = ord(char) - ascii_offset
23             x = (a_inv * (y - b)) % 26
24             result += chr(x + ascii_offset)
25         else:
26             result += char
27    return result

```

Listing 2.2 – affine.py - Fonctions principales

L'inverse modulaire est calculé avec l'algorithme d'Euclide étendu via `pow(a, -1, 26)`.

2.3 Chiffrement de Vigenère

2.3.1 Principe

Le chiffrement de Vigenère est un chiffrement polyalphabétique utilisant un mot-clé. Chaque lettre du texte est décalée selon la lettre correspondante de la clé :

$$E(x_i) = (x_i + k_{i \bmod n}) \mod 26$$

2.3.2 Code source

```

1 def vigenere_encrypt(text: str, key: str) -> str:
2     result = ""
3     key = key.upper()
4     key_length = len(key)
5     key_as_int = [ord(k) - ord('A') for k in key]
6     for i, char in enumerate(text):
7         if char.isalpha():
8             key_index = i % key_length
9             key_shift = key_as_int[key_index]
10            if char.isupper():
11                result += chr((ord(char) - ord('A') + key_shift)
12                             % 26 + ord('A'))
13            else:
14                result += chr((ord(char) - ord('a') + key_shift)
15                             % 26 + ord('a'))
16        else:
17            result += char
18    return result
19
20 def vigenere_decrypt(text: str, key: str) -> str:
21     # Meme logique avec soustraction du decalage
22     ...

```

Listing 2.3 – vigenere.py

2.4 Chiffrement de Hill

2.4.1 Principe

Le chiffrement de Hill utilise l'algèbre linéaire. Le texte est découpé en vecteurs de taille n et multiplié par une matrice-clé K de taille $n \times n$ modulo 26. Le déchiffrement utilise la matrice inverse $K^{-1} \bmod 26$.

2.4.2 Code source

```

1 def hill_encrypt(text: str, key_matrix: np.ndarray) -> str:
2     n = key_matrix.shape[0]
3     text = ''.join(c for c in text if c.isalpha()).lower()
4     if len(text) % n != 0:
5         text += 'x' * (n - len(text) % n) # padding
6     vectors = []
7     for i in range(0, len(text), n):
8         vector = [ord(text[i + j]) - ord('a') for j in range(n)]
9         vectors.append(vector)
10    result = ""
11    for vector in vectors:
12        encrypted_vector = np.dot(key_matrix, vector) % 26
13        for num in encrypted_vector:
14            result += chr(int(num) + ord('a'))
15    return result
16
17 def hill_decrypt(text: str, key_matrix: np.ndarray) -> str:

```

```

18     inverse_key = matrix_mod_inverse(key_matrix, 26)
19     return hill_encrypt(text, inverse_key)

```

Listing 2.4 – Hill.py - Chiffrement

2.5 Chiffrement de Playfair

2.5.1 Principe

Le chiffrement de Playfair utilise une matrice 5×5 construite à partir d'un mot-clé. Le texte est découpé en bigrammes, et chaque paire est chiffrée selon trois règles : même ligne, même colonne, ou rectangle.

2.5.2 Code source

```

1 def create_playfair_matrix(key: str) -> List[List[str]]:
2     key = key.upper().replace('J', 'I')
3     alphabet = []
4     for char in key + 'ABCDEFGHIJKLMNOPQRSTUVWXYZ':
5         if char not in alphabet and char.isalpha():
6             alphabet.append(char)
7     matrix = []
8     for i in range(0, 25, 5):
9         matrix.append(alphabet[i:i + 5])
10    return matrix

```

Listing 2.5 – Playfair.py - Construction de la matrice

2.6 One-Time Pad (OTP)

2.6.1 Principe

Le One-Time Pad est le seul chiffrement théoriquement incassable. Il utilise une clé aléatoire de même longueur que le message et effectue un XOR bit à bit.

2.6.2 Code source

```

1 import os
2
3 def otp_encrypt(message: str) -> tuple[bytes, bytes]:
4     message_bytes = message.encode("utf-8")
5     key = os.urandom(len(message_bytes))
6     ciphertext = bytes(m ^ k for m, k in zip(message_bytes, key))
7     return ciphertext, key
8
9 def otp_decrypt(ciphertext: bytes, key: bytes) -> str:
10    message_bytes = bytes(c ^ k for c, k in zip(ciphertext, key))
11    return message_bytes.decode("utf-8")

```

Listing 2.6 – OTP.py

2.7 Substitution Aléatoire

2.7.1 Principe

Chaque lettre de l'alphabet est remplacée par une autre lettre selon une table de substitution générée aléatoirement.

2.7.2 Code source

```
1 import random, string
2 from typing import Dict
3
4 def generate_random_substitution() -> Dict[str, str]:
5     alphabet = string.ascii_lowercase
6     shuffled = list(alphabet)
7     random.shuffle(shuffled)
8     return {alphabet[i]: shuffled[i] for i in range(len(alphabet))}
9
10 def substitution_encrypt(text, substitution_table):
11     result = ""
12     for char in text.lower():
13         if char in substitution_table:
14             result += substitution_table[char]
15         else:
16             result += char
17     return result
```

Listing 2.7 – Substitution_Aleatoire.py

Chapitre 3

Outils de Cryptanalyse

3.1 Analyse Fréquentielle

3.1.1 Principe

L'analyse fréquentielle compare la fréquence des lettres d'un texte chiffré aux fréquences théoriques de la langue (ici le français). Cela permet de casser les chiffrements par substitution mono-alphabétique.

3.1.2 Code source

```
1 FRENCH_FREQ = {  
2     'A': 7.636, 'B': 0.901, 'C': 3.260, 'E': 14.715, ...  
3 }  
4  
5 def analyse_frequentielle(texte):  
6     texte = normalize_text(texte)  
7     lettres = [c for c in texte if c in string.ascii_lowercase]  
8     total = len(lettres)  
9     compte = Counter(lettres)  
10    frequencies = {l: (compte[l] / total) * 100 for l in compte}  
11    return frequencies
```

Listing 3.1 – AnalyseFrequentielle.py

Le programme génère aussi un histogramme comparatif avec `matplotlib`.

3.2 Indice de Coïncidence

3.2.1 Principe

L'indice de coïncidence (IC) mesure la probabilité que deux lettres choisies au hasard soient identiques :

$$IC = \frac{\sum_{i=1}^{26} n_i(n_i - 1)}{N(N - 1)}$$

3.2.2 Code source

```
1 def indice_coincidence(text):
2     text = normalize_text(text)
3     letters = [c for c in text if c in string.ascii_lowercase]
4     N = len(letters)
5     if N <= 1:
6         return 0
7     counts = Counter(letters)
8     somme = sum(n * (n - 1) for n in counts.values())
9     return somme / (N * (N - 1))
```

Listing 3.2 – IndiceDeCoincidence.py

3.3 Test de Kasiski

3.3.1 Principe

Le test de Kasiski permet d'estimer la longueur de la clé d'un chiffrement de Vigenère en recherchant les séquences répétées et en calculant le PGCD des distances entre ces répétitions.

3.3.2 Code source

```
1 def find_repeated_sequences(ciphertext, min_length=3):
2     sequences = {}
3     for i in range(len(ciphertext) - min_length + 1):
4         seq = ciphertext[i:i + min_length]
5         if seq in sequences:
6             sequences[seq].append(i)
7         else:
8             sequences[seq] = [i]
9     gaps = []
10    for seq, positions in sequences.items():
11        if len(positions) > 1:
12            for j in range(1, len(positions)):
13                gaps.append(positions[j] - positions[j - 1])
14    return gaps
15
16 def find_key_length(ciphertext):
17     gaps = find_repeated_sequences(ciphertext)
18     if not gaps:
19         return None
20    return reduce(gcd, gaps)
```

Listing 3.3 – TestDeKasiski.py - Fonctions clés

Chapitre 4

Chiffrement Symétrique (Synchrone)

4.1 DES (Data Encryption Standard)

4.1.1 Principe

Le DES est un algorithme de chiffrement par blocs de 64 bits avec une clé de 56 bits. Il utilise un réseau de Feistel à 16 rounds. Notre implémentation inclut également le 2DES, 3DES et DESX.

4.1.2 Architecture

L'algorithme se décompose en :

1. **Permutation initiale (IP)** : réarrangement des 64 bits
2. **16 rounds de Feistel** : chaque round applique expansion, XOR avec sous-clé, S-Boxes et permutation P
3. **Permutation finale (FP)** : inverse de IP

4.1.3 Code source

```
1 def feistel(right, subkey):
2     expanded = permute(right, E) # 32 -> 48 bits
3     xored = xor_bits(expanded, subkey)
4     sbbox_output = []
5     for i in range(8):
6         block = xored[i*6:(i+1)*6]
7         row = (block[0] << 1) | block[5]
8         col = (block[1] << 3) | (block[2] << 2) | (block[3] << 1) | block[4]
9         sbbox_output.extend(
10             [int(b) for b in format(S_BOXES[i][row][col], '04b')]
11         )
12     return permute(sbbox_output, P)
```

Listing 4.1 – DES.py - Fonction de Feistel

```
1 def des_encrypt_block(block_64bits, subkeys):
2     state = permute(block_64bits, IP)
3     L, R = state[:32], state[32:]
4     for i in range(16):
5         new_L = R
```

```

6         new_R = xor_bits(L, feistel(R, subkeys[i]))
7         L, R = new_L, new_R
8     return permute(R + L, FP)

```

Listing 4.2 – DES.py - Chiffrement d'un bloc

```

1 def _3des_encrypt(plaintext, key1, key2, key3=None):
2     """3DES: E_k1(D_k2(E_k3(plaintext)))"""
3     if key3 is None:
4         key3 = key1 # Mode 2-cles: K1=K3
5     encrypted = des_encrypt(plaintext, key3)
6     decrypted = des_decrypt(encrypted, key2)
7     return des_encrypt(decrypted, key1)

```

Listing 4.3 – DES.py - Triple DES

4.1.4 Variantes implémentées

- **DES simple** : chiffrement ECB avec blocs de 64 bits
- **2DES** : double application du DES avec deux clés
- **3DES (EDE)** : triple application Encrypt-Decrypt-Encrypt
- **DESX** : ajout de whitening keys (XOR avant/après DES)

4.2 AES (Advanced Encryption Standard)

4.2.1 Principe

L'AES est le standard actuel de chiffrement symétrique. Il opère sur des blocs de 128 bits avec des clés de 128, 192 ou 256 bits. Les opérations principales sont : SubBytes, ShiftRows, MixColumns et AddRoundKey.

4.2.2 Code source

Notre implémentation a évolué pour utiliser la bibliothèque professionnelle `pycryptodome`, offrant des performances optimales et le support des modes ECB, CBC et CTR, tout en corrigeant les vulnérabilités de notre première approche manuelle.

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
4
5 def aes_encrypt_cbc(data, key, iv=None):
6     """AES mode CBC."""
7     if isinstance(data, str):
8         data = data.encode('utf-8')
9     if isinstance(key, str):
10        key = key.encode('utf-8')[:32].ljust(32, b'\x00')
11    if iv is None:
12        iv = get_random_bytes(AES.block_size)
13
14    cipher = AES.new(key, AES.MODE_CBC, iv=iv)
15    padded = pad(data, AES.block_size)
16    ciphertext = cipher.encrypt(padded)
17    return iv + ciphertext

```

Listing 4.4 – AES.py - Chiffrement CBC

4.2.3 Modes implémentés

- **ECB** : chaque bloc chiffré indépendamment
- **CBC** : chaînage avec XOR du bloc précédent
- **CTR** : mode compteur (chiffrement de flux)

4.3 RC4 (Rivest Cipher 4)

4.3.1 Principe

RC4 est un chiffrement de flux (stream cipher). Il génère un flux pseudo-aléatoire (keystream) via deux algorithmes : KSA (Key Scheduling Algorithm) et PRGA (Pseudo-Random Generation Algorithm).

4.3.2 Code source

```

1 class RC4:
2     def __init__(self, key):
3         if isinstance(key, str):
4             key = key.encode('utf-8')
5         self.key = key
6         self.S = list(range(256))
7         self._ksa(key)
8
9     def _ksa(self, key):
10        """Key Scheduling Algorithm"""
11        key_length = len(key)
12        j = 0
13        for i in range(256):
14            j = (j + self.S[i] + key[i % key_length]) % 256
15            self.S[i], self.S[j] = self.S[j], self.S[i]
16
17    def _prga(self):
18        """Pseudo-Random Generation Algorithm"""
19        self._i = (self._i + 1) % 256
20        self._j = (self._j + self.S[self._i]) % 256
21        self.S[self._i], self.S[self._j] = self.S[self._j], self.S[self._i]
22        return self.S[(self.S[self._i] + self.S[self._j]) % 256]
23
24    def encrypt(self, data):
25        if isinstance(data, str):
26            data = data.encode('utf-8')
27        self._i = 0; self._j = 0
28        self.S = list(range(256))
29        self._ksa(self.key)
30        result = bytearray()
31        for byte in data:
32            result.append(byte ^ self._prga())
33        return bytes(result)

```

Listing 4.5 – RC4.py - Algorithmes KSA et PRGA

4.3.3 Variantes implémentées

- **RC4 Standard** : implémentation classique
- **RC4A** : variante avec deux S-boxes
- **RC4Drop** : saute les N premiers bytes du keystream
- **RC4WithHash** : clé dérivée via SHA-256

Chapitre 5

Chiffrement Asymétrique

5.1 RSA

5.1.1 Principe

RSA repose sur la difficulté de factoriser de grands nombres. On choisit p, q premiers, $n = p \cdot q$, $\phi = (p - 1)(q - 1)$, e tel que $\gcd(e, \phi) = 1$ et $d = e^{-1} \mod \phi$.

$$\text{Chiffrement : } c = m^e \mod n \quad (5.1)$$

$$\text{Déchiffrement : } m = c^d \mod n \quad (5.2)$$

5.1.2 Code source

Le code a évolué d'une implémentation pédagogique basique vers l'utilisation de `cryptography` pour intégrer le padding sécurisé OAEP, essentiel contre les attaques sur RSA textbook, ainsi qu'une approche de chiffrement hybride pour de larges volumes de données.

```
1 from cryptography.hazmat.primitives.asymmetric import rsa, padding
2 from cryptography.hazmat.primitives import hashes
3 from cryptography.hazmat.backends import default_backend
4
5 def generate_rsa_key_pair(key_size: int = 2048):
6     private_key = rsa.generate_private_key(
7         public_exponent=65537,
8         key_size=key_size,
9         backend=default_backend()
10    )
11    return private_key, private_key.public_key()
12
13 def rsa_encrypt_oaep(message: bytes, public_key) -> bytes:
14     return public_key.encrypt(
15         message,
16         padding.OAEP(
17             mgf=padding.MGF1(algorithm=hashes.SHA256()),
18             algorithm=hashes.SHA256(),
19             label=None
20         )
21    )
```

Listing 5.1 – RSA.py - Génération et Chiffrement OAEP

5.2 ElGamal

5.2.1 Principe

ElGamal est basé sur le problème du logarithme discret. On choisit un nombre premier p , un générateur g , une clé privée a et la clé publique $A = g^a \mod p$.

$$\text{Chiffrement : } c_1 = g^k \mod p, \quad c_2 = m \cdot A^k \mod p \quad (5.3)$$

$$\text{Déchiffrement : } m = c_2 \cdot (c_1^a)^{-1} \mod p \quad (5.4)$$

5.2.2 Code source

```

1 def encrypt(message, public_key):
2     p, g, A = public_key
3     cipher = []
4     for char in message:
5         m = ord(char)
6         k = random.randint(2, p - 2)
7         c1 = pow(g, k, p)
8         c2 = (m * pow(A, k, p)) % p
9         cipher.append((c1, c2))
10    return cipher
11
12 def decrypt(cipher, private_key, p):
13     a = private_key
14     message = ""
15     for c1, c2 in cipher:
16         s = pow(c1, a, p)
17         s_inv = mod_inverse(s, p) # Fermat: a^(p-2) mod p
18         m = (c2 * s_inv) % p
19         message += chr(m)
20    return message

```

Listing 5.2 – ElGamal.py - Chiffrement/Déchiffrement

5.3 Diffie-Hellman

5.3.1 Principe

Diffie-Hellman permet à deux parties d'établir un secret partagé via un canal non sécurisé. Avec p premier et g racine primitive :

1. Alice choisit a , calcule $A = g^a \mod p$
2. Bob choisit b , calcule $B = g^b \mod p$
3. Secret partagé : $s = B^a \mod p = A^b \mod p$

5.3.2 Code source

```

1 def generate_dh_parameters():
2     p = 23; g = 5 # parametres pedagogiques
3     return p, g
4

```

```
5 def generate_public_key(g, private_key, p):
6     return pow(g, private_key, p)
7
8 def compute_shared_secret(other_public, private_key, p):
9     return pow(other_public, private_key, p)
10
11 def derive_aes_key(shared_secret):
12     """Derive une cle AES-256 via SHA-256"""
13     secret_bytes = str(shared_secret).encode('utf-8')
14     return hashlib.sha256(secret_bytes).digest()
```

Listing 5.3 – Diffie_Hellman.py

5.4 Elliptic Curve Cryptography (ECC)

5.4.1 Principe

La cryptographie sur les courbes elliptiques (ECC) offre un niveau de sécurité équivalent à RSA mais avec des clés beaucoup plus petites. Elle repose sur la difficulté du logarithme discret sur le groupe des points d'une courbe elliptique sur un corps fini.

5.4.2 Code source

```
1 from cryptography.hazmat.primitives.asymmetric import ec
2 from cryptography.hazmat.primitives import serialization, hashes
3 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
4
5 def generate_ecc_keys():
6     private_key = ec.generate_private_key(ec.SECP256R1())
7     public_key = private_key.public_key()
8     return private_key, public_key
9
10 def derive_shared_key(private_key, peer_public_key):
11     shared_key = private_key.exchange(ec.ECDH(), peer_public_key)
12     derived_key = HKDF(
13         algorithm=hashes.SHA256(),
14         length=32,
15         salt=None,
16         info=b'handshake data',
17     ).derive(shared_key)
18     return derived_key
```

Listing 5.4 – ECC.py

Chapitre 6

Fonctions de Hachage

6.1 Principe général

Les fonctions de hachage produisent une empreinte de taille fixe à partir d'un message de taille quelconque. Propriétés essentielles : résistance aux collisions, résistance à la pré-image, effet avalanche.

6.2 MD5

```
1 import hashlib
2
3 def md5_hash(data):
4     """Calcule le hash MD5 d'une donn e."""
5     if isinstance(data, str):
6         data = data.encode('utf-8')
7     return hashlib.md5(data).hexdigest()
```

Listing 6.1 – MD5.py - Calcul du Hash

Le script a été amélioré pour inclure des démonstrations de l'effet d'avalanche (environ 50% des bits modifiés pour un changement d'un seul bit en entrée) et explique les vulnérabilités aux collisions de MD5. MD5 produit un hash de 128 bits (32 caractères hexadécimaux). **Note :** MD5 est considéré comme non sécurisé pour les usages cryptographiques.

6.3 SHA-256 et SHA-512

Le script SHA256.py propose désormais une implémentation **pédagogique complète** de l'algorithme (padding Merkle-Damgård, compression en 64 tours, fonctions Σ, γ) ainsi que l'utilisation de `hashlib` pour vérifier l'intégrité des résultats.

```
1 def sha256_compress(block: bytes, state: list) -> list:
2     """Fonction de compression sur un bloc de 512 bits."""
3     # Extension du message: 16 mots -> 64 mots
4     w = [0] * 64
5     for i in range(16):
6         w[i] = int.from_bytes(block[i*4:(i+1)*4], 'big')
7     for i in range(16, 64):
```

```
8         w[i] = (gamma1(w[i-2]) + w[i-7] + gamma0(w[i-15]) + w[i-16]) & 0
          xFFFFFFFF
9
10     # ... boucle principale de compression (64 tours) ...
11     # Mise a jour de l'etat final
12     return new_state
```

Listing 6.2 – SHA256.py - Algorithme de compression manuel

SHA-256 produit 256 bits, SHA-512 produit 512 bits. Ces fonctions font partie de la famille SHA-2 et sont largement utilisées en production.

6.4 HMAC

```
1 import hmac, hashlib
2
3 def hmac_hash():
4     word = input("Enter the word to hash with HMAC: ")
5     key = input("Enter the secret key for HMAC: ")
6     hmac_obj = hmac.new(
7         key.encode('utf-8'),
8         word.encode('utf-8'),
9         hashlib.sha256
10    )
11    print(f"HMAC (SHA256) Hash: {hmac_obj.hexdigest()}")
```

Listing 6.3 – HMAC.py

HMAC combine une fonction de hachage avec une clé secrète pour vérifier l'intégrité et l'authenticité des messages.

Chapitre 7

Signatures Numériques

7.1 Principe

Une signature numérique garantit l'authenticité et la non-répudiation d'un message. L'expéditeur signe avec sa clé privée, le destinataire vérifie avec la clé publique.

7.2 Signature RSA

```
1 class RSASignature:
2     def __init__(self, key_size=2048):
3         self.private_key = rsa.generate_private_key(
4             public_exponent=65537, key_size=key_size)
5         self.public_key = self.private_key.public_key()
6
7     def sign_pkcs1v15(self, message):
8         return self.private_key.sign(
9             message, padding.PKCS1v15(), hashes.SHA256())
10
11     def verify_pkcs1v15(self, message, signature):
12         try:
13             self.public_key.verify(
14                 signature, message,
15                 padding.PKCS1v15(), hashes.SHA256())
16             return True
17         except Exception:
18             return False
```

Listing 7.1 – Signatures.py - RSA PKCS#1 v1.5 et PSS

7.3 Signature ElGamal

Implémentation complète avec génération de nombres premiers via `sympy`, recherche de racines primitives, signature et vérification :

$$y^r \cdot r^s \equiv g^{H(m)} \pmod{p}$$

7.4 Signature DSA

Le DSA (Digital Signature Algorithm) est un standard NIST utilisant le problème du logarithme discret dans un sous-groupe.

7.5 Signature ECDSA

L'ECDSA utilise les courbes elliptiques (courbe SECP256R1) pour des signatures plus compactes avec le même niveau de sécurité.

7.6 Benchmark

Le programme inclut un benchmark comparatif mesurant les temps de signature et vérification pour chaque algorithme.

Chapitre 8

Canal de Communication Sécurisé

8.1 Architecture

Le canal sécurisé implémente une communication Client/Serveur utilisant :

- **RSA 2048** pour l'authentification et l'échange de clés
- **AES-256 GCM** pour le chiffrement des messages
- **RSA-PSS** pour la signature des messages
- **RSA-OAEP** pour le chiffrement de la clé AES

8.2 Génération des clés

```
1 def generate_keys(name):
2     """Genere une paire de cles RSA 2048"""
3     private_key = rsa.generate_private_key(
4         public_exponent=65537,
5         key_size=2048,
6         backend=default_backend()
7     )
8     public_key = private_key.public_key()
9     # Sauvegarde en format PEM
10    with open(f"{name}_private_key.pem", "wb") as f:
11        f.write(private_key.private_bytes(
12            encoding=serialization.Encoding.PEM,
13            format=serialization.PrivateFormat.PKCS8,
14            encryption_algorithm=serialization.NoEncryption()
15        ))
```

Listing 8.1 – keygen.py

8.3 Utilitaires cryptographiques

```
1 class CryptoTools:
2     @staticmethod
3     def encrypt_message(plaintext_bytes, peer_public_key):
4         """Chiffre avec AES-256 GCM"""
5         aes_key = AESGCM.generate_key(bit_length=256)
6         cipher = AESGCM(aes_key)
7         nonce = os.urandom(12)
```

```
8     ciphertext = cipher.encrypt(nonce, plaintext_bytes, None)
9     # Chiffrer la cle AES avec RSA-OAEP
10    encrypted_key = peer_public_key.encrypt(
11        aes_key,
12        padding.OAEP(
13            mgf=padding.MGF1(algorithm=hashes.SHA256()),
14            algorithm=hashes.SHA256(), label=None
15        )
16    )
17    return encrypted_key, nonce, ciphertext
```

Listing 8.2 – crypto_utils.py - Chiffrement hybride

8.4 Protocole de communication

1. Le message est **signé** avec la clé privée RSA de l'expéditeur (RSA-PSS)
2. Une clé AES-256 aléatoire est générée
3. Le message est **chiffré** avec AES-256-GCM
4. La clé AES est **chiffrée** avec la clé publique RSA du destinataire (OAEP)
5. Le paquet JSON contient : clé chiffrée, nonce, texte chiffré, signature

8.5 Exécution

```
1 # Terminal 1 : Serveur
2 python secure_Channel/keygen.py      # Generation des cles
3 python secure_Channel/secure_channel_server.py
4
5 # Terminal 2 : Client
6 python secure_Channel/secure-channel-client.py
```

Lancement du canal securise

Le mode debug affiche les données chiffrées/déchiffrées pour visualiser le processus de sécurisation.

Chapitre 9

Interface Graphique (GUI)

9.1 Présentation de l'application

Le projet intègre une interface graphique complète et ergonomique (`gui.py`) développée avec les bibliothèques `tkinter` et `ttk`. Nommée **CryptoSuite Complete**, cette application centralise l'ensemble des algorithmes étudiés lors des TP 1 à 6 dans une interface unifiée.

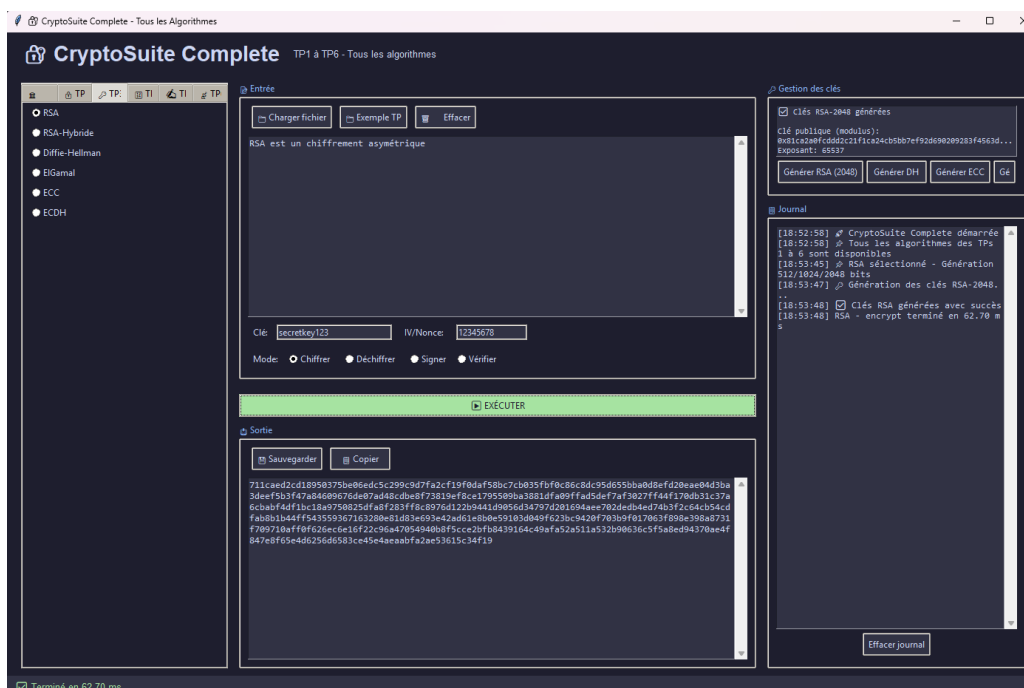


FIGURE 9.1 – Interface principale de CryptoSuite Complete

9.2 Architecture de l'interface

L'interface est structurée en trois panneaux principaux pour offrir une expérience utilisateur fluide :

- **Panneau Gauche (Algorithmes)** : Un système d'onglets permet de naviguer facilement entre les différentes catégories d'algorithmes :

- *TP1 - Classique* : César, Vigenère, Hill, OTP, Playfair, Rail Fence, et outils d'analyse fréquentielle.
- *TP2 - Symétrique* : RC4, DES, 3DES, AES (128/192/256), Twofish, Serpent, RC6, MARS.
- *TP3 - Asymétrique* : RSA, RSA-Hybride, Diffie-Hellman, ElGamal, ECC, ECDH.
- *TP4 - Hachage* : MD5, SHA-256, SHA-512, HMAC, effet d'avalanche.
- *TP5 - Signatures* : RSA-PSS, DSA, ECDSA, ElGamal.
- *TP6 - Applications* : Simulations de protocoles sécurisés comme Bluetooth, Wi-Fi WPA2/WPA3, Vote électronique.
- **Panneau Central (Entrée/Sortie)** : Il permet à l'utilisateur de saisir son texte, de charger un fichier, ou d'utiliser des exemples intégrés. L'utilisateur peut y définir ses propres clés et vecteurs d'initialisation (IV/Noncé), choisir le mode d'opération (Chiffrer, Déchiffrer, Signer, Vérifier), puis exécuter l'algorithme sélectionné. Les résultats sont affichés dans une zone de sortie avec des options de sauvegarde et de copie.
- **Panneau Droit (Gestion des Clés et Journal)** : Ce panneau intègre un gestionnaire de clés interactif permettant de générer en un clic des paires de clés asymétriques complexes (RSA-2048, paramètres Diffie-Hellman, clés ECC P-256, ElGamal). Un journal d'événements (log) intégré trace toutes les actions effectuées par l'utilisateur.

9.3 Apport Pédagogique

La **CryptoSuite Complete** constitue un puissant outil pédagogique. Chaque algorithme affiche des infobulles explicatives sur ses particularités (ex : "Attaque à clair connu" pour Hill, "Vulnérabilités WEP" pour RC4). L'interface permet ainsi de comparer instantanément la sortie et la complexité des différents chiffrements sans avoir à manipuler la ligne de commande.

Chapitre 10

Questions et Réponses

10.1 Limites pratiques de l'OTP (One-Time Pad)

Question : L'OTP est théoriquement parfait mais pratiquement inutilisable, citer des obstacles concrets.

Réponse : Bien que l'OTP offre un secret parfait de Shannon, son utilisation en pratique se heurte à plusieurs obstacles majeurs :

- **Taille de la clé :** La clé doit être au moins aussi longue que le message à chiffrer ($|K| = |M|$).
- **Génération de l'aléa :** La clé doit être générée de manière parfaitement aléatoire (entropie maximale) et non pseudo-aléatoire.
- **Distribution des clés :** Échanger de manière sécurisée une clé de la taille du message est logistiquement complexe, car cela nécessite un canal de communication tout aussi sécurisé.
- **Usage unique :** La clé ne doit être utilisée qu'une seule fois. Toute réutilisation compromet immédiatement le système (two-time pad).

10.2 Impact de la longueur de la clé et lien avec l'OTP

Question : En quoi la longueur de la clé impacte-t-elle la sécurité ? Que se passe-t-il quand $|K| = |M|$? (lien avec OTP)

Réponse : La longueur de la clé détermine la taille de l'espace des clés. Plus la clé est longue, plus il y a de combinaisons possibles, rendant les attaques par force exhaustive irréalisables avec la puissance de calcul actuelle ou future.

Lorsque la longueur de la clé est égale à la longueur du message ($|K| = |M|$), comme dans le cas de l'OTP (avec une clé aléatoire et à usage unique), on atteint la **sécurité parfaite**. Dans cette configuration, chaque chiffré peut correspondre à n'importe quel texte clair possible de la même longueur. L'attaquant, même avec une puissance de calcul infinie, ne peut déduire aucune information sur le message d'origine à partir du chiffré.

10.3 Vulnérabilité du chiffrement de Hill

Question : Pourquoi Hill est-il vulnérable à l'attaque à clair connu même pour de grandes matrices ?

Réponse : Le chiffrement de Hill est basé sur l'algèbre linéaire ($C = K \cdot P \pmod{26}$).

Dans une attaque à clair connu, l'attaquant n'a pas besoin de tester toutes les matrices possibles par force brute. S'il intercepte suffisamment de paires pour former une matrice P inversible de taille $n \times n$ (où n est la taille de la matrice clé K), il peut simplement résoudre l'équation :

$$K = C \cdot P^{-1} \pmod{26}$$

La résolution de ce système d'équations (inversion de matrice) se fait en temps polynomial. Ainsi, même pour de très grandes matrices, retrouver la clé K est très rapide.

10.4 Chiffrement RSA et Padding OAEP

Question : Pourquoi RSA ne peut pas chiffrer directement un message de taille arbitraire ? Qu'est-ce que le padding OAEP apporte par rapport à RSA textbook ?

Réponse : Taille du message : L'algorithme RSA opère dans l'anneau des entiers modulo n . Par définition mathématique, la valeur numérique du message m doit être strictement inférieure à n . Chiffrer un message plus grand nécessiterait de le découper en blocs, ce qui est inefficace et vulnérable structurellement (similaire au mode ECB). C'est pourquoi RSA est généralement utilisé pour chiffrer des petites données (comme des clés AES symétriques).

Apport de l'OAEP : Le "RSA textbook" (pur) est déterministe (un même message donne toujours le même chiffré) et malléable. L'OAEP (Optimal Asymmetric Encryption Padding) apporte deux propriétés fondamentales :

- **Randomisation :** Il introduit un aléa cryptographique (salt) avant le chiffrement, rendant RSA probabiliste (sémantiquement sûr).
- **Contre la malléabilité :** Il structure mathématiquement le message avec des fonctions de hachage, empêchant les attaques par chiffré choisi. Toute modification du chiffré sera détectée lors du déchiffrement.

10.5 Sélection de Rijndael pour l'AES face à Serpent

Question : L'algorithme Serpent a obtenu la meilleure note pour la sécurité mais n'a pas été retenu. Quel critère a fait pencher la balance vers Rijndael ?

Réponse : Lors du concours AES organisé par le NIST, Serpent offrait effectivement la marge de sécurité la plus élevée (notamment grâce à ses 32 rounds). Cependant, Rijndael a été sélectionné grâce à son **excellent équilibre global**, et plus particulièrement pour ses **performances** et son **agilité** :

- **Performances :** Rijndael était significativement plus rapide en implémentation logicielle et matérielle. La complexité de Serpent impactait lourdement les débits de chiffrement.
- **Agilité :** Rijndael fonctionnait extrêmement bien sur un large panel de plateformes, des processeurs performants aux puces intégrées (smartcards) disposant de très peu de mémoire (RAM) et de capacités limitées.

Chapitre 11

Conclusion

Ce projet nous a permis d'implémenter et de comprendre en profondeur les principaux algorithmes cryptographiques, depuis les chiffrements classiques jusqu'aux protocoles modernes de communication sécurisée.

11.1 Résumé des implémentations

Catégorie	Algorithmes	Fichiers
Classiques	César, Affine, Vigenère, Hill, Playfair, OTP	7 fichiers
Cryptanalyse	Fréquentielle, IC, Kasiski	3 fichiers
Symétrique	DES/2DES/3DES/DESX, AES-128/192/256, RC4	3 fichiers
Asymétrique	RSA, ElGamal, Diffie-Hellman, ECC	4 fichiers
Hachage	MD5, SHA-256, SHA-512, HMAC	4 fichiers
Signatures	RSA, ElGamal, DSA, ECDSA	1 fichier
Canal sécurisé	RSA+AES-GCM Client/Serveur	4 fichiers
Interface	GUI Complète (CryptoSuite) avec Tkinter	1 fichier (<code>gui.py</code>)

TABLE 11.1 – Résumé des algorithmes implémentés

11.2 Compétences acquises

- Compréhension des fondements mathématiques de la cryptographie
- Implémentation des algorithmes de chiffrement classiques et modernes
- Mise en pratique des protocoles de sécurité (signatures, échange de clés)
- Développement d'une application réseau sécurisée